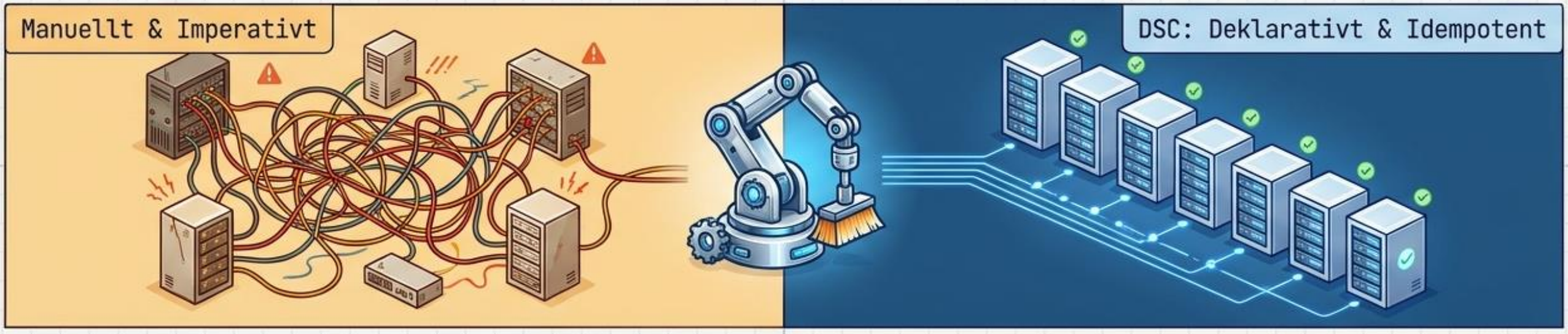
PowerShell DSC (Desired State Configuration)

Arseniy Buzyan

“Städroboten” för dina servrar

DSC är en plattform som automatiskt säkerställer att dina **Windows-system** alltid befinner sig i exakt det tillstånd du har definierat.



Deklarativt (HUR, inte VAD)

Du beskriver målbilden ("IIS ska vara installerat, en specifik fil ska finnas"). DSC räknar ut vägen dit.

Idempotent (Säker upprepning)

Konfigurationen kan köras tusentals gånger. Är systemet redan korrekt? Då gör DSC ingenting.

Från manuellt kaos till automatisk konsekvens

Konfigurationsglidning

(Servrar ändras över tid och blir unika snöflingor).

Manuella fel

(Glömda steg och slarv vid nattliga konfigurationer).

”Det funkar på min dator”

(Omöjligt att återskapa specifika miljöer).







Självläkande:

DSC återställer automatiskt servern till det exakta önskade tillståndet.

Maskinell Konsekvens:

Allt görs automatiskt och exakt likadant varje gång.

Absolut Standardisering:

Samma kodbaserad konfiguration rullas ut överallt.

Från Verkligheten – DSC I Svenska IT-miljöer

DSC används för att automatisera och standardisera Windows-serverkonfiguration i produktion.

Exempel på användningsområden

- Ny serverinstallation DSC installerar automatiskt alla roller och funktioner (t.ex. IIS, DNS, DHCP)
- Säkerhetsstandarder DSC säkerställer att brandväggsregler, tjänster och registerinställningar är korrekta
- Webbplatsdrift DSC håller IIS-webbplatser igång med rätt inställningar
- Azure-miljöer DSC används med Azure Automation State Configuration för att hantera både lokala servrar och moln-VM:ar
- Active Directory DSC-moduler för deployment och konfiguration av Active Directory Domain Services finns tillgängliga via DSC Community
- SQL Server DSC-resurser för deployment och konfiguration av Microsoft SQL Server finns i SqlServerDsc-modulen

Varför detta är viktigt: Enligt en kurs från Skillsoft är DSC en central komponent i Microsoft DevOps-lösningar (AZ-400) för att upptäcka och åtgärda konfigurationsglidning i Windows-miljöer .

DSC Arkitektur – Tre Huvudkomponenter

DSC består av tre komponenter som samverkar för att hålla dina servrar i önskat tillstånd.

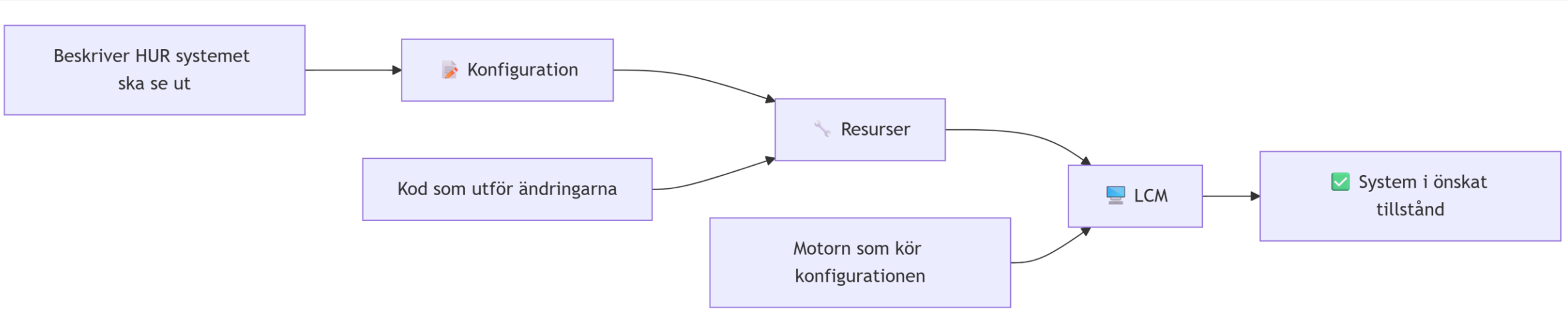


Diagram – DSC arkitektur

Arkitekturen: DSC:s Tre Huvudkomponenter



1. Konfiguration (Ritningen):

PowerShell-skriptet som deklarerar HUR systemet ska se ut.

2. Resurser (Verktygen):

Koden som under huven faktiskt utför ändringarna (t.ex. skapar mappar, installerar IIS).

3. Local Configuration Manager (LCM) (Motorn):

Den inbyggda Windows-komponenten som regelbundet pollar systemet och applicerar konfigurationen.

Generationsskiftet: DSC v2 vs. DSC v3 (2026)

En komplett omskrivning från grunden.

Aspekt	DSC v2 (Klassisk)	DSC v3 (Modern)
Format	MOF-filer	JSON eller YAML
Plattform	Endast Windows	Cross-platform (Windows, Linux, macOS)
Resursspråk	`PowerShell`	`PowerShell`, Python, Bash, Go, Rust
Körning	LCM (inbyggd tjänst)	Fristående kommandoradsverktyg (`dsc.exe`)

⚠ **PARADIGMSKIFTE: LCM FÖRSVINNER.**

DSC v3 har ingen inbyggd bakgrundstjänst som kontinuerligt pollar systemet. Du måste framöver själv schemalägga `dsc.exe` (t.ex. via Task Scheduler) för kontinuerlig drift.

Framtidssäkring med ‘Version Pinning’



```yaml
# Lås DSC-versionen
directives:
  version: '=3.2.0'

resources:
  - name: os
    type: Microsoft/OSInfo
    requireVersion: '^1.0' 1.0.x eller högre, men ALDRIG 2.0
```

- **Vad är det?** Låser exakt **vilken version** av **DSC-motorn** och dess **resurser** som ska användas.
- **Varför?** Utan pinning används alltid senaste versionen. Uppdateringar riskerar att ’bryta’ konfigurationen. **Semantisk versionering** garanterar exakt samma beteende varje körning.

Från arkitektur till kod

Konfiguration (Ritningen)

```
Configuration InstallIIS {
    ...
}
```

Beskriver hela systemets målbild i ett samlat skript.

Resurs (Verktyget)

```
WindowsFeature IIS {
    Name = "Web-Server"
}
```

Det specifika kodblocket inuti konfigurationen som anropar installationslogiken.

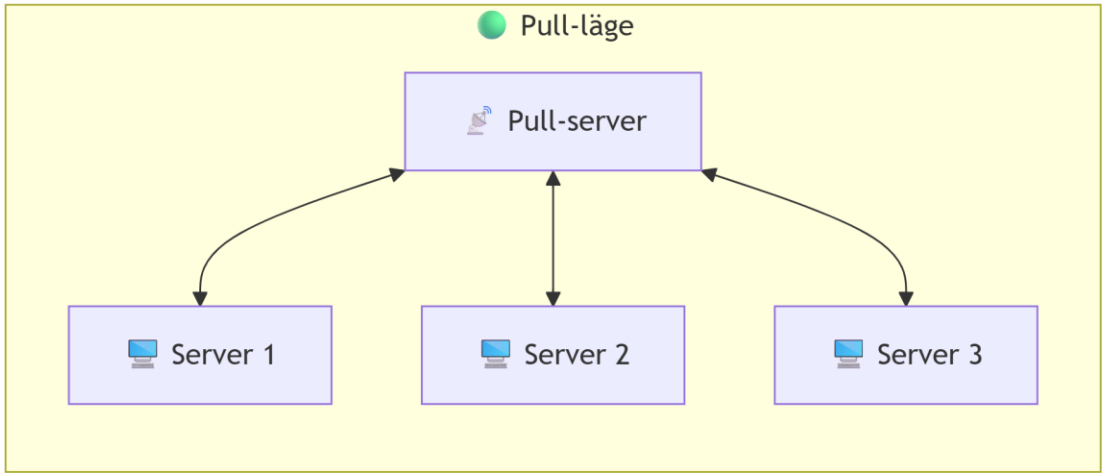
LCM (Motorn)

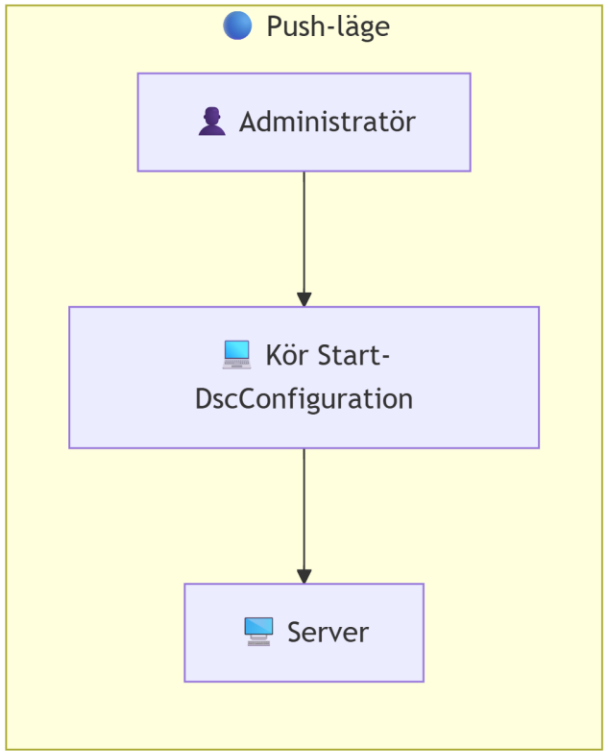
```
Start-DscConfiguration
    -Path .\InstallIIS
```

Kommandot som väcker LCM-motorn för att läsa och applicera konfigurationen.

DSC kan användas i två olika lägen: Push och Pull. Välj rätt för din situation!

Push vs Pull – Två Sätt Att Använda DSC





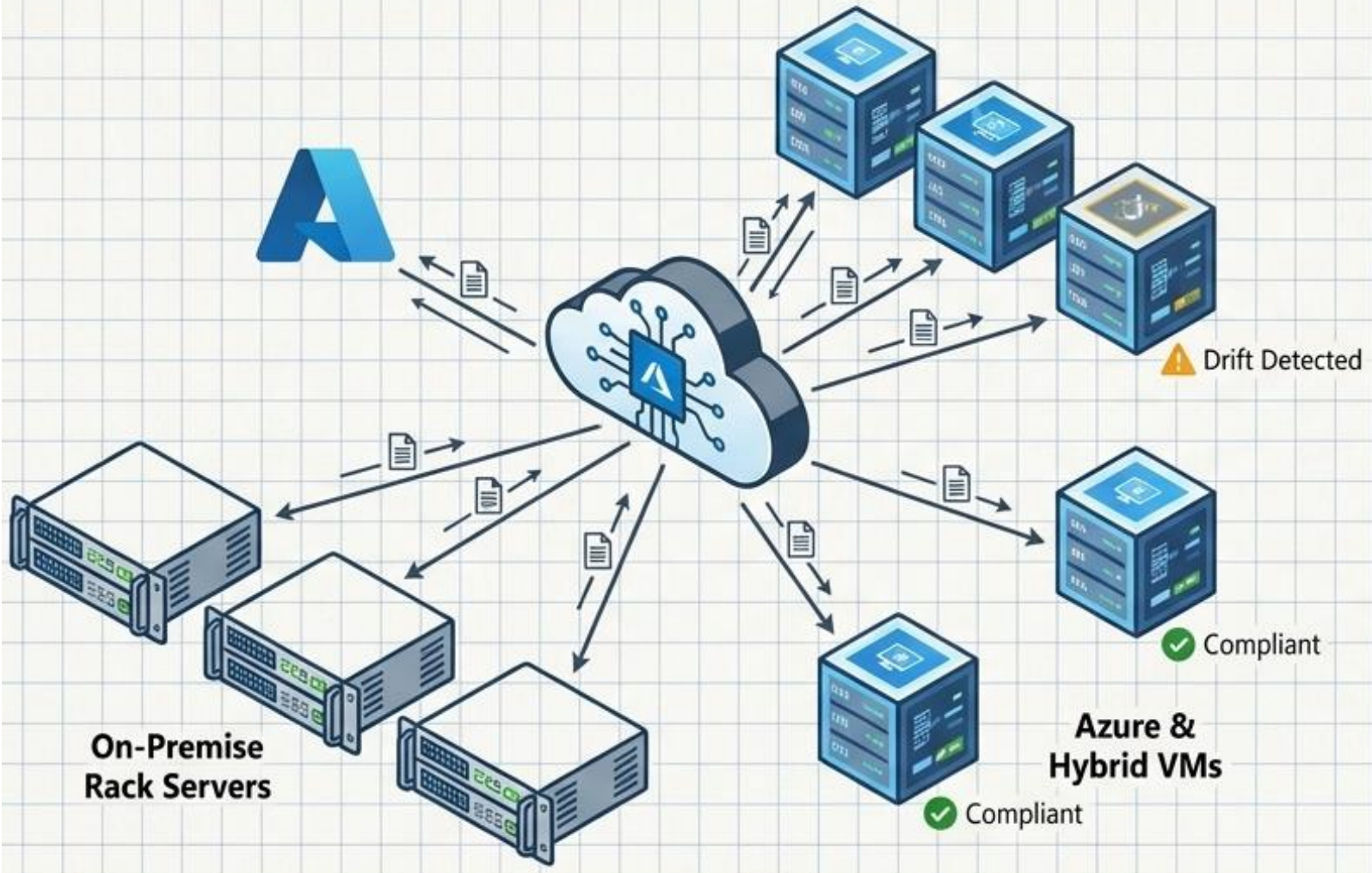
Jämförelse :

Enligt Microsoft: Azure Automation inkluderar en hanterad DSC-pull-server i molnet som gör det enkelt att lagra DSC-konfigurationer och ansluta hanterade datorer.

Aspekt	Push-läge	Pull-läge
Vem initierar?	Administratören skickar konfigurationen	Servrar hämtar konfigurationen från en central server
Användning	Testning, felsökning, enstaka servrar	Produktion, många servrar, central hantering
Kommando	Start-DscConfiguration -Path .\Config	Servrar pollar automatiskt (default 15 min)
Azure-integration	–	Azure Automation DSC Pull Server
Resurshantering	Administratör måste installera resurser manuellt	DSC kan installera resurser automatiskt

Azure Automation State Configuration

Den rekommenderade Pull-arkitekturen för moderna DevOps-flöden.



Central Hantering:
En enda lagringsplats i molnet för alla dina DSC-konfigurationer.

Skalbarhet:
Kapacitet att hantera tusentals servrar från ett gränssnitt.

Rapportering:
Omedelbar insyn – vilka noder uppfyller kraven och vilka har driftat?

Hybridstöd:
Fungerar sömlöst för Azure VM:ar, lokala servrar och Azure Automanage.

Kräver: Windows 8.1 / Server 2012 R2+, WinRRM tillgängligt, och nätverksåtkomst över TCP 443.

Krav för DSC med Azure Automation

- Enligt Microsoft: Azure Automation State Configuration och Azure Automanage Machine Configuration används för att implementera DSC i moderna DevOps-flöden.
- **Operativsystem** Windows 8.1 eller senare, eller Windows Server 2012 R2 eller senare
- **WinRM** Windows Remote Management måste vara tillgängligt
- **Nätverk** TCP 443, *.azure-automation.net, *.agentsvc.azure-automation.net

DSC-resurser – Byggblocken

Resurser är de byggblock som DSC använder för att utföra ändringar.

File	Hanterar filer och mappar	File MyFile { DestinationPath = "C:/temp/file.txt"; Ensure = "Present" }
Service	Hanterar Windows-tjänster	Service W3SVC { Name = "W3SVC"; State = "Running" }
WindowsFeature	Installerar Windows-funktioner	WindowsFeature IIS { Name = "Web-Server"; Ensure = "Present" }
Registry	Hanterar registervärden	Registry MyKey { Key = "HKLM:\SOFTWARE\MyApp"; ValueData = "1.0" }

DSC Community-resurser

- Enligt DSC Community: 99+ open-source DSC-resursmoduler finns tillgängliga för att hantera allt från SQL Server till Hyper-V och Exchange Server .
- **ActiveDirectoryDsc** Deployment och konfiguration av Active Directory
- **NetworkingDsc** Konfiguration av nätverksinställningar
- **SqlServerDsc** Deployment och konfiguration av SQL Server
- **ComputerManagementDsc** Datorkonfiguration (domän, schemalagda uppgifter, minne, tidszon)
- **SharePointDsc** Deployment och hantering av SharePoint-farmar

Version Pinning – Nytt i DSC v3.2.0

"Version pinning" = Lås fast vilken version av DSC och resurser som ska användas.

Varför är detta viktigt?

Utan pinning använder DSC alltid den senaste versionen

En ny version kan innehålla ändringar som bryter din konfiguration

Med pinning får du exakt samma beteende oavsett när du kör

Exempel – pinning i YAML

Enligt 4sysops: ^1.0 betyder "alla versioner från 1.0.0 upp till men inte inklusive 2.0.0". Om DSC inte hittar en kompatibel version uppstår ett fel.

yaml

```yaml
# Lås DSC-versionen
directives:
  version: '=3.2.0'

resources:
- name: os
  type: Microsoft/OSInfo
  requireVersion: '^1.0'  # 1.0.x eller högre, men inte 2.0
  properties: {}
```

Workshop – Din Första DSC-konfiguration (Klassisk DSC v2)

Mål: Skapa din första DSC-konfiguration som säkerställer att en tjänst är igång.

1 **Skapa:**
Skriv PowerShell-skriptet till vänster.

2 **Generera:**
Kör **EnsureService** i konsolen för att kompilera .mof-filen.

3 **Applicera:**
Kör `Start-DscConfiguration -Path .\EnsureService -Wait -Verbose.`

4 **Verifiera:**
Kolla status med `Get-Service -Name W3SVC.`

Question Box
Diskussionsfråga: Vad händer om du kör Steg 3 en gång till?
(Svar: Ingenting. DSC är idempotent!)

Steg 1 – Skapa EnsureService.ps1:

```powershell
Configuration EnsureService {
    Node "localhost" {
        Service W3SVC {
            Name = "W3SVC"
            State = "Running"
            StartupType = "Automatic"
        }
    }
}
```

Steg 2 – Generera MOF

- `powershell`
- `EnsureService`

Steg 3 – Applicera

```powershell
Start-DscConfiguration -Path .\EnsureService -Wait -Verbose
```

Steg 4 – Testa

```powershell
Get-Service -Name W3SVC
```

Diskussionsfråga: Varför är DSC idempotent? Vad händer om du kör `Start-DscConfiguration` en gång till?

Sammanfattning

Vad DSC är: En plattform för att automatiskt säkerställa att system alltid är i önskat tillstånd

Tre komponenter: Konfiguration, Resurser, Local Configuration Manager

Push vs Pull: Push = administratör skickar konfiguration , Pull = servrar hämtar från central server

DSC v2 vs v3: DSC v3 är en komplett omskrivning – ingen LCM, JSON/YAML-konfiguration

Version Pinning: Lås versioner med version: '=3.2.0' och requireVersion

DSC Community: 99+ open-source-resurser för Active Directory, SQL Server, SharePoint, Hyper-V

Azure Automation: Rekommenderad plattform för DSC i produktion

Quiz

https://forms.gle/dRKvj7qTZcHVWiGt7